

RESEARCH ARTICLE

Fourier neural networks as function approximators and differential equation solvers

Marieme Ngom  | Oana Marin

Mathematics and Computer Science
Division, Argonne National Laboratory,
Lemont, Illinois, USA

Correspondence

Marieme Ngom, 9700 S. Cass Avenue,
Lemont, IL 60439, USA.
Email: mngom@anl.gov

Funding information

U.S. Department of Energy, Office of
Science, Advanced Scientific Computing
Research; FASTMath Institute

Abstract

We present a Fourier neural network (FNN) that can be mapped directly to the Fourier decomposition. The choice of activation and loss function yields results that replicate a Fourier series expansion closely while preserving a straightforward architecture with a single hidden layer. The simplicity of this network architecture facilitates the integration with any other higher-complexity networks, at a data pre- or postprocessing stage. We validate this FNN on naturally periodic smooth functions and on piecewise continuous periodic functions. We showcase the use of this FNN for modeling or solving partial differential equations with periodic boundary conditions. The main advantages of the current approach are the validity of the solution outside the training region, interpretability of the trained model, and simplicity of use.

KEYWORDS

differential equations, Fourier decomposition, neural networks

1 | INTRODUCTION

The past decade has seen revolutionary advances in machine learning (ML) techniques, in particular through deep learning [8]. The increasing availability of data through data-collecting initiatives and technologies has made the use of ML ubiquitous in image recognition and finance. One popular class of ML models is neural networks built to mimic the human brain. In the past 60 years a plethora of neural network architectures such as convolutional neural networks [16], recurrent neural networks [11], and autoencoders [30] were introduced in the literature. Depending on the task at hand, a specific architecture can prove more advantageous than another. For function approximation, for example, feedforward networks have been widely used [1, 5, 12]. They are multilayered

networks where the information travels from the input to the output only in the forward direction. Each layer of a feedforward network is composed of nodes linked across layers through weights and biases and activated through an activation function, for example, the ReLU, the hyperbolic tangent (tanh), logistic, or sigmoid functions [9, 22].

The objective of this work is to build a special feedforward network that approximates periodic functions and seek periodic solutions to partial differential equations (PDEs). We present a particular type of feedforward networks, Fourier neural networks (FNNs), which are shallow neural networks with a sinusoidal activation function. The terminology FNN is indicative of the neural network design since it mimics the Fourier decomposition as first introduced in [32]. However, neural networks with sinusoidal activation functions were first investigated in

[7]. In [19], the author presented a FNN that specializes in regression and classification tasks. The authors in [34] provide a comprehensive comparative study of existing FNN architectures, and recently another approach of constructing a Fourier operator was developed [18]. The main originality of FNNs is the nature of the activation function, which incorporates sinusoidal functions and is different from the traditional ones (ReLU, sigmoid function, etc.). Many approximation theorems are available for these traditional activation functions. In [5, 12], it was proven that shallow neural networks with squashing activation functions such as the logistic or sigmoid functions could approximate any Borel measurable functions to any desired order of accuracy. In [17], the authors showed that multilayered neural networks with nonpolynomial activation functions could approximate any function up to $O(1/N)$. In [1], the author gave universal approximation bounds for superpositions of a sigmoidal function for functions whose first moment of the magnitude distribution of the Fourier transform is bounded. With respect to FNNs, the authors in [7] proved an approximation theorem for a squashed cosine activation function.

The current work introduces a new methodology to approximate analytic and piecewise continuous periodic functions using FNNs. The methodology is then used as groundwork to address the second objective of this paper, which is to seek periodic solutions to PDEs. Periodicity is ubiquitous in the physical sciences (periodicity of seasons in climate science, etc.) and in the computational sciences (periodic boundary conditions in large-scale applications) and is thus a relevant computational regime. Periodic solutions of differential equations occur naturally, for example, for equations in electronics or oscillatory systems. For differential equations with periodic boundary conditions, solutions are periodic as well. Furthermore, there is a growing interest in solving differential equations using neural networks. The authors in [33] introduced the deep Galerkin method (DGM), a mesh-free algorithm that effectively solves high-dimensional PDEs by using a deep neural network (DNN). To train the neural network, they used a loss function that incorporates the differential equations and the boundary and initial conditions. The DGM algorithm reduces the computational cost of traditional approaches, such as the finite difference method, by randomly sampling points in the computational domain instead of meshing it. In [25, 26], the authors developed a physics informed neural network (PINN) that aims at both solving and learning PDEs from data when they are not known. They showcased their network's performance by effectively solving and learning the Schrödinger, Burgers, Navier–Stokes, Korteweg–de Vries, and Kuramoto–Sivashinsky equations. A comprehensive discussion on how to solve the Poisson equation

and the steady-state Navier–Stokes equation based on [25–27] is provided in [6]. The authors in [13] modified existing iterative PDE solvers with a DNN to accelerate their convergence speed. They trained their neural network on a specific geometry, thus being able to generalize its performance to a diverse range of boundary conditions and geometries while significantly improving the speedup as compared with the performance of the original solver.

In the first part of this paper, we present the architecture of our FNN which uses the full cosine as an activation function (instead of the squashed one). Furthermore, we embed the periodicity information in the loss function, which ensures, upon convergence, that the network will approximate the Fourier series outside the training interval. We restrict ourselves to the approximation of low-frequency continuous periodic functions. As shown in [23], a significant drawback in training FNNs is that the optimization can stagnate in a local minimum because of the high number of oscillations. Limiting ourselves to low-frequency modes and incorporating the periodicity in the loss function allow us to overcome that difficulty. We verify the results numerically on different functions and exploit the constructed FNN to recover low-frequency Fourier coefficients. One bottleneck of ML is the difficulty of preserving the “learned” parameters outside the training domain. Given the choice of both activation and loss function, the parameters learned by the FNN are also valid outside the training region.

The second part of this paper focuses on using the constructed FNN to seek periodic solutions of differential equations, which is a novel use of this network design. Here we follow [27, 33] and incorporate the equations in the loss function to obtain a physics informed Fourier neural network (PIFNN). We show the performance of the built PIFNN on a range of PDEs, such as the Poisson equation and the heat equation.

The paper is organized as follows. In Section 2 we construct the FNN and provide an initialization strategy for the weights and biases of the network. In Section 3 we present different numerical simulations that showcase the advantages of the built network. In Section 4 we modify the constructed network so it becomes a PIFNN that aims at seeking periodic solutions of a range of differential equations. In Section 5 we summarize our conclusions and briefly discuss avenues for future research.

2 | FNNS AS FUNCTION APPROXIMATORS

A neural network can be seen as a function approximator with a number of inputs M , a number of hidden layers

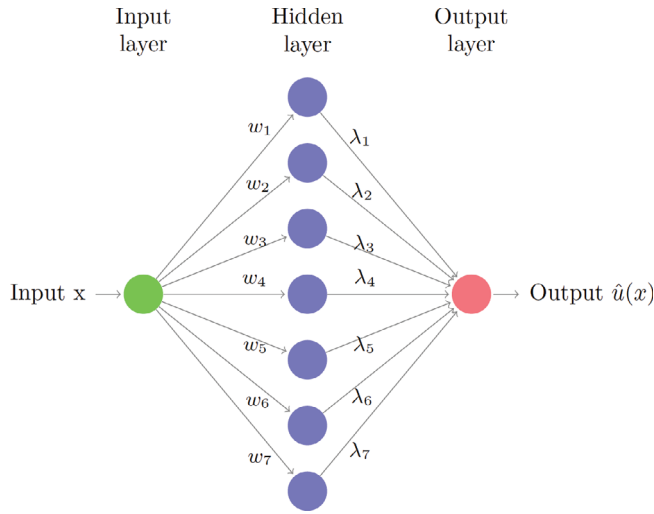


FIGURE 1 Fully connected neural network (biases not shown) with a single hidden layer and a one-dimensional input $x \in \mathbf{R}$

that are composed of nodes, and a number of outputs L . In this work, we focus on shallow neural networks with one input, one output and N nodes in the hidden layer (see Figure 1). The goal is to approximate real-valued periodic functions. The output $\hat{u}$ of such neural networks can be written as

$$\hat{u}(x) = \phi_0 + \sum_{k=1}^N \lambda_k \xi(w_k x + \phi_k), \quad (1)$$

where $x \in \mathbf{R}$ is the input, $w = (w_k, k = 1, \dots, N)$ and $\lambda = (\lambda_k, k = 1, \dots, N)$ are the weights of the neural network, ξ the activation function, and $\phi = (\phi_k, k = 0, \dots, N)$ its biases.

The discrete Fourier series representation $S_N u$ of a T -periodic function $u \in L^2(\mathbf{R})$ is

$$S_N u(x) = \frac{a_0}{2} + \sum_{n=1}^N a_n \cos(n\omega x) + b_n \sin(n\omega x), \quad (2)$$

where $x \in \left[-\frac{T}{2}, \frac{T}{2}\right]$, $\omega = \frac{2\pi}{T}$, and a_n and b_n are the Fourier coefficients of u . Equation (1) can be rewritten in the following reduced (phase shift) form:

$$S_N u(x) = \frac{a_0}{2} + \sum_{n=1}^N c_n \cos(n\omega x + \psi_n), \quad (3)$$

where for $n \geq 1$, $c_n = \sqrt{a_n^2 + b_n^2}$, and $\psi_n = \arctan 2\left(-\frac{b_n}{a_n}\right)$.

Therefore, by taking the activation function to be $x \mapsto \cos(\omega x)$ in Equation (1), one obtains a formulation that resembles the Fourier decomposition. The input to hidden layer weights, w_k , mimics the nodes, n ; the hidden

layer to output weights, λ_k , approximates the Fourier coefficients, c_n ; the biases of the hidden layer, ϕ_k , correspond to the phase shifts, ψ_k ; and the bias of the output, ϕ_0 , corresponds to half the zeroth Fourier coefficient, a_0 . As we empirically show below, however, the equivalence is not straightforward; and one needs to train the network with a specific loss function in order to obtain satisfactory results. In the rest of this paper, we restrict ourselves to 2-periodic functions, which means the activation function is $x \mapsto \cos(\pi x)$. This restriction however is made only to facilitate the design of a robust neural network and derive a reliable initialization that assures fast convergence. If the data-set is not 2-periodic it can be easily mapped to a 2-periodic set as a data processing step.

We note that Equation (3) can be regarded as an interpretable neural network given its translatable relationship with Equation (2). However no explicit translation is necessary since the node values obtained are already the modes of the Fourier series and can be directly used.

2.1 | Loss function

The goal of this network design is to approximate the Fourier series $S_N u$ of u . To this end, we define the loss function as

$$L(\phi, w, \lambda) = \|\hat{u} - u\|_2^2 + \alpha_1 \|\lambda\|^2 + \alpha_2 \|w\|^2, \quad (4)$$

where

- the first term $\|\hat{u} - u\|_2^2$ ensures that the output of the neural network approximates the target function u , and can be regarded as a least-square estimation for obtaining Fourier coefficients [24],
- the last two terms, $\alpha_1 \|\lambda\|^2$ and $\alpha_2 \|w\|^2$, are regularization terms to avoid overfitting of the neural network. Choosing a L^2 norm for the regularization parameters causes the weights to decay asymptotically to zero, while a L^1 regularization might decrease them to exactly zero. We reserve the choice of the regularization norm for Section 3.

However, this loss function will not provide an approximation of the Fourier series of a function unless the function is an exact combination of cosines or sines. For example, attempting to fit a piecewise continuous periodic function such as

$$u(x) = x^2, \quad x \in (-(2k+1), (2k+1)), \quad k \in \mathbf{N}$$

with one neuron in the hidden layer leads to the output

$$\hat{u}(x) = \phi_0 - \lambda_1 \cos(\pi w_1 x),$$

where $\frac{\lambda_1 \pi^2 w_1^2}{2!} \approx 1$, $\lambda_1 \approx \phi_0 \approx 1$, and $(\pi w_1)^{2l} \ll 1$ for $l > 1$. Let us scrutinize the Taylor expansion of the chosen activation function:

$$\begin{aligned} \cos(\pi w_1 x) &= 1 + \sum_{l=1}^{\infty} (-1)^l \frac{(\pi w_1)^{2l} x^{2l}}{(2l)!} \\ &= 1 - \frac{\pi^2 w_1^2}{2!} x^2 + o(w_1^3). \end{aligned}$$

This indicates that if we seek the Fourier series of x^2 , which is the first order term in the Taylor expansion of the activation function, the network may converge rapidly to

$$x^2 \approx (1 - \cos(\pi w_1 x)) \frac{2!}{\pi^2 w_1^2} \approx \phi_0 - \lambda_1 \cos(\pi w_1 x).$$

This, however is not the Fourier expansion of x^2 , which will be further studied in Section 3. To alleviate such pitfalls and obtain the Fourier representation of the target function, we need to enforce the weights from the input to the hidden layer converge to integer values. This can be achieved by solving a mixed-integer optimization problem, which is out of the scope of this paper and we solve this issue here by introducing a novel family of loss functions as follows:

$$\begin{aligned} L(\phi, w, \lambda) &= \|\hat{u}(x) - u(x)\|_2^2 + \alpha_1 \|\lambda\|^2 + \alpha_2 \|w\|^2 \\ &\quad + \alpha_3 (\|\hat{u}(x+T) - \hat{u}(x)\|_2^2) \\ &\quad + \alpha_4 (\|\hat{u}(x-T) - \hat{u}(x)\|_2^2). \end{aligned} \quad (5)$$

This choice forces the output of the neural network to be periodic; and as we will show, this loss function preserves the periodicity of the solution outside the training interval.

2.2 | Weights and biases initialization

A proper weight initialization can significantly improve a neural network's performance in the training phase, especially when dealing with DNNs. When training a DNN, one is often faced with the vanishing/exploding gradients phenomenon [8], which causes the training not to converge to a good solution. The authors in [9] proposed an initialization method using the logistic activation function that circumvents the vanishing/exploding gradients issue. In essence, they proved that the variance of each layer's outputs needs to be equal to the variances of its inputs.

Furthermore, the gradient variances should be equal as the layers are traversed in the reverse direction. He et al. [10] proposed an initialization strategy for the ReLU activation function. In [15], this was extended to nonlinear activation functions differentiable in the vicinity of the origin.

Even though we consider a shallow neural network, initialization is essential for accelerating the training phase, and we propose a similar strategy. We initialize the biases to zero and determine appropriate weights for initialization. To this end, we revisit a few properties of the mean and variance of a random variable, denoted here by X to differentiate it from its realization x . Let x be the input of our FNN and $\{h_k\}_{k=1, \dots, N}$ the hidden layer nodes, see Figure 1. For brevity, we denote by y the output of the network and reserve $\hat{u}(x)$ for the Fourier series representation. Since the biases are taken to be zero, we have

$$h_k = \cos(\pi w_k x) \text{ and } y = \sum_{k=1}^N \lambda_k \cos(\pi w_k x).$$

We assume the network is trained using M samples x of a random variable X , drawn from a uniform distribution on the period interval $\mathcal{U}(T)$, here we choose $T = 2$, that is, $\mathcal{U}(T) = [-1, 1]$. A uniform distribution is a suitable choice for the input layer to assure no particular preference is given to any data. Therefore the random variable X has zero mean, that is, $\mu(X) = 0$, and variance $\sigma^2(X) = 1/3$. A refresher on basic probability theory is given in Appendix A, here we define only the mean and variance as

$$\mu(X) = \int_{-\infty}^{+\infty} x f(x) dx, \quad (6)$$

$$\sigma^2(X) = \mu(X^2) - (\mu(X))^2. \quad (7)$$

For the hidden layer we have the set of weights; w_k and for the output layer λ_k , as in Figure 1. Both sets are drawn from normal distributions $\mathcal{N}(0, m^2)$ and $\mathcal{N}(0, v^2)$, respectively, and we denote them by random variables W_k and Λ_k , respectively, that is

$$W_k \sim \mathcal{N}(0, m^2),$$

and

$$\Lambda_k \sim \mathcal{N}(0, v^2).$$

Remark 1. Requiring that the variances at each layer of the network are equal during the first forward pass, that is

$$\sigma^2(H_k) = \sigma^2(Y), \quad (8)$$

assures that the network initialization yields good convergence properties.

This remark allows us to determine values for m and v , and consequently appropriate distributions for the weights w_k and v_k , as will be shown.

2.2.1 | Initialization of the hidden layer weights w_k

We first compute the mean of the node k of the hidden layer. To this end, we use the following theorem [28], also known as the law of the unconscious statistician (LOTUS).

Theorem 1. *Let X, Y be continuous random variables with a joint density function $f_{(X,Y)}$, and let h be a continuous function of two variables such that*

$$\int_{\mathbb{R}^2} |h(x, y)| f_{(X,Y)}(x, y) dx dy < +\infty, \quad \text{then}$$

$$\mu(h(X, Y)) = \int_{\mathbb{R}^2} h(x, y) f_{(X,Y)}(x, y) dx dy.$$

Knowing that the joint probability distribution of the two independent random variables W_k and X is

$$f_{(W_k,X)}(x, y) = \frac{1}{2} \cdot \frac{1}{m\sqrt{2\pi}} e^{-\frac{w_k^2}{2m^2}},$$

we obtain (using $h(w_k, x) = \cos(\pi w_k x)$ in the above theorem)

$$\begin{aligned} \mu(H_k) &= \mu(\cos(\pi W_k X)) \\ &= \int_{-1}^1 \int_{-\infty}^{+\infty} \frac{1}{2} \cos(\pi w_k x) \frac{1}{m\sqrt{2\pi}} e^{-\frac{w_k^2}{2m^2}} dw_k dx. \end{aligned} \quad (9)$$

Let $I_1(x) = \int_{-\infty}^{+\infty} \cos(\pi w_k x) \frac{1}{m\sqrt{2\pi}} e^{-\frac{w_k^2}{2m^2}} dw_k$. By differentiating under the integral sign we obtain

$$I_1'(x) = - \int_{-\infty}^{+\infty} \pi w_k \sin(\pi w_k x) \frac{1}{m\sqrt{2\pi}} e^{-\frac{w_k^2}{2m^2}} dw_k.$$

After integrating by parts $I_1'(x)$, we find I_1 satisfies the differential equation

$$\begin{aligned} I_1'(x) + \pi^2 m^2 x I_1(x) &= 0, \\ I_1(0) &= \int_{-\infty}^{+\infty} \frac{1}{m\sqrt{2\pi}} e^{-\frac{w_k^2}{2m^2}} dw_k = \frac{m}{|m|}, \end{aligned}$$

which admits the unique solution

$$I_1(x) = \frac{m}{|m|} e^{-\frac{1}{2}\pi^2 m^2 x^2} = e^{-\frac{1}{2}\pi^2 m^2 x^2}.$$

We can now represent the mean as

$$\mu(H_k) = \int_{-1}^1 \frac{1}{2} e^{-\frac{1}{2}\pi^2 m^2 x^2} dx,$$

which, after integration, gives

$$\mu(H_k) = \frac{1}{m\sqrt{2\pi}} \operatorname{erf}\left(\frac{m\pi}{\sqrt{2}}\right), \quad (10)$$

where erf is the error function.

To compute the variance of H_k , we recall that $\sigma^2(H_k) = \mu(H_k^2) - (\mu(H_k))^2$. Using the trigonometric identity

$$h_k^2 = \cos^2(\pi w_k x) = \frac{1}{2} + \frac{\cos(2\pi w_k x)}{2},$$

we obtain

$$\mu(H_k^2) = \int_{-1}^1 \frac{1}{2} \int_{-\infty}^{+\infty} \frac{1}{2} (1 + \cos(2\pi w_k x)) \frac{1}{m\sqrt{2\pi}} e^{-\frac{w_k^2}{2m^2}} dw_k dx.$$

After integration we have the mean of the random variable squared

$$\mu(H_k^2) = \frac{1}{2} + \frac{1}{4\sqrt{2\pi}} \frac{\operatorname{erf}(\sqrt{2}m\pi)}{m}.$$

The variance of H_k is now readily available from Equation (7) as

$$\sigma^2(H_k) = \frac{1}{2} + \frac{1}{4\sqrt{2\pi}} \frac{\operatorname{erf}(\sqrt{2}m\pi)}{m} - \left(\frac{1}{m\sqrt{2\pi}} \operatorname{erf}\left(\frac{m\pi}{\sqrt{2}}\right) \right)^2. \quad (11)$$

Equation (11) establishes a closed form expression for the connection between $\sigma^2(H_k)$ and the variance m . To this end let us denote the relation between the layers of the network, stated in Remark 1, as $s = \sigma^2(H_k) = \sigma^2(Y)$, and refer to s as the network initialization constant. In Figure 2 we evaluate $\sigma^2(H_k)$, as provided by Equation (11), to identify the behavior of the variances m as a function of the network initialization constant s . We note that for values of the variance m larger than approximately $\sqrt{5}$, identified via a red vertical line in Figure 2a, the solution s is asymptotically constant. In Figure 2b we illustrate a possible set of values for the variance m given the constant s , and observe that the asymptotic behavior of s in Equation (11) implies that values $s \gtrsim 0.52$ are not feasible.

2.2.2 | Initialization of the output layer weights λ_k

Since Λ_k and $\cos(W_k X)$ are independent, we can write the mean and variance of the output layer Y

$$\mu(Y) = \sum_{k=1}^N \mu(\Lambda_k) \mu(\cos(W_k X)),$$

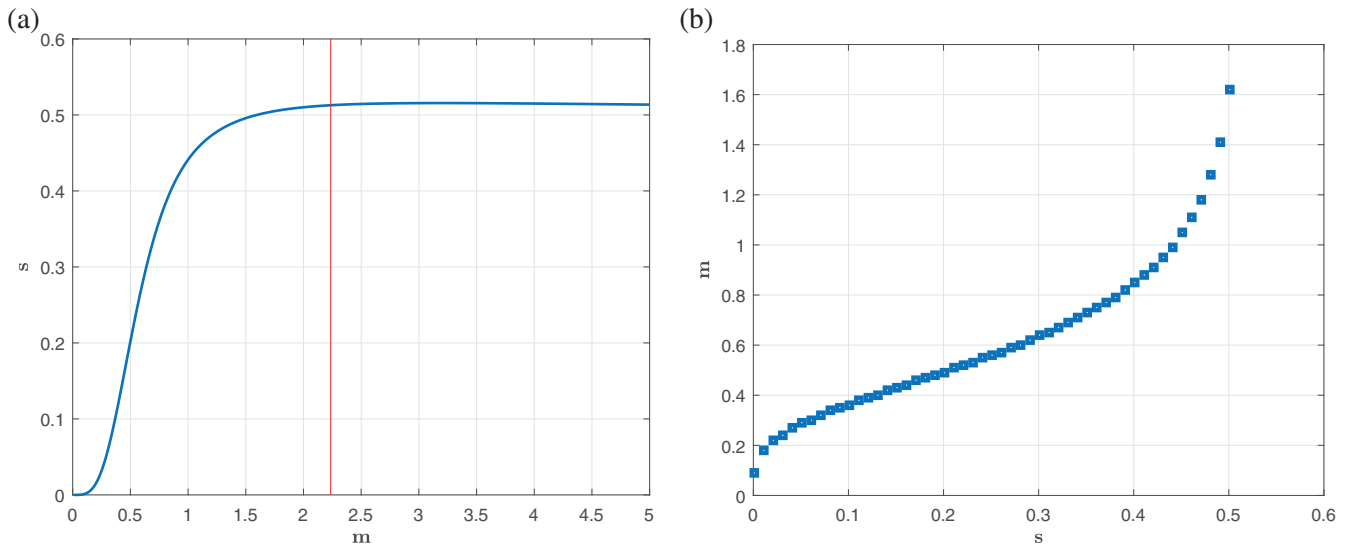


FIGURE 2 Analysis of the choice of hidden layer weights and their variance. (A) Right hand side $s = \sigma^2(H_k)$ of Equation (11). (b) Solution set for Equation (11)

and

$$\sigma^2(Y) = \sum_{k=1}^N \sigma^2(\Lambda_k) (\sigma^2(\cos(W_k X)) + \mu^2(\cos(W_k X))) + \mu^2(\Lambda_k) \sigma^2(\cos(W_k X)),$$

where $\sigma^2(\Lambda_k) = v^2$ and $\mu(\Lambda_k) = 0$. This leads to

$$\sigma^2(Y) = N \left[v^2 \left[\frac{1}{2} + \frac{1}{4\sqrt{2\pi}} \frac{\text{erf}(\sqrt{2}m\pi)}{m} \right] \right], \quad (12)$$

where N represents the number of nodes in the hidden layer chosen at the initialization step. Following again Remark 1 with $s = \sigma^2(Y) = \sigma^2(H_k)$, we obtain

$$v^2 = \frac{s}{N \left(\frac{1}{2} + \frac{1}{4\sqrt{2\pi}} \frac{\text{erf}(\sqrt{2}m\pi)}{m} \right)}. \quad (13)$$

Without yet making any choice for the network initialization constant s , we illustrate in Figure 3a the relationship between the variance v and the constant s , and in Figure 3b the relationship between variances v and m . Note that in Equation (13) the variance v depends on N , the nodes in the hidden layer, for which we select a set of values $N = [4, 8, 12, 16, 20]$. Furthermore in Figure 3b we observe that too small values of m , up to approximately $m < 0.2$ lead to almost zero variances in the output layer. This is not a desirable feature and will inform our choices. Also both in Figure 3a,b we note that an increasing number of nodes N leads to lower values of the variances v , as a function of m .

From this analysis we conclude that good values for m are in the region where m is the largest possible, however not larger than approximately 2.5, according to Figure 2a. In practice we choose $m = \sqrt{5}$, that is, $W_k \sim \mathcal{N}(0, 5)$. Plugging in $m = \sqrt{5}$ in Equation (11) evaluates to $\sigma^2(H_k) \approx .5128$; and one further substitution in Equation (12) yields $v^2 = 0.9703/N$.

The choice of a fixed value for m is made here to provide a general framework for the numerical studies that follow. However in particular cases, such as choices of inner nodes N , different periodic intervals etc., better convergence properties may be achieved for different feasible values of m .

3 | NUMERICAL STUDIES

To assess the effectiveness of the FNN in approximating periodic functions, we perform numerical studies on both periodic smooth functions and functions periodically extensible via continuity. First, we approximate analytic periodic functions, given as linear combinations of sines and cosines. High accuracy is expected on such functions since we have direct access to the Fourier modes. Second, we recover low-frequency Fourier coefficients of piecewise continuous periodic functions, such as $x \mapsto |x|$ and $x \mapsto x^2$. These functions are of particular interest since they are prevalent in fields such as acoustics and electronics [2]. As expected, non-smooth periodic functions place an additional strain on the training of the FNN, either in terms of iterations to convergence or in meeting the tolerance threshold. However, the results

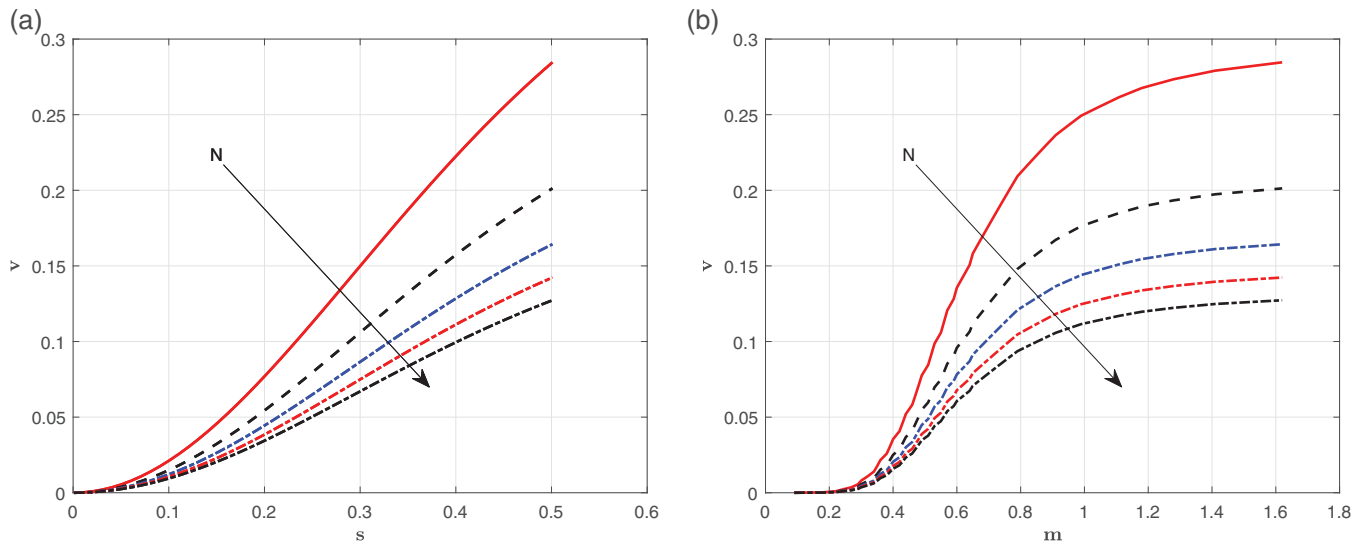


FIGURE 3 Analysis of output layer weights and their variance, browsing a set of nodes $N = [4, 8, 12, 16, 20]$ chosen for the hidden layer. (a) Right-hand side v of Equation (13) as function of $s = \sigma^2(Y) = \sigma^2(H_k)$. (b) Relationship between the variance m of the hidden layer, and the output layer variance v from Equation (13)

TABLE 1 Number of iterations, value of the loss function at convergence, and optimal weights and biases of the FNN to approximate $f(x) = \cos(\pi x) + \sin(\pi x)$, $k = 1, \dots, 4$ with a L^2 regularization

Number of iterations			189
Loss function (upon convergence)			$2e - 4$
w_k	ϕ_k	λ_k	ϕ_0
1.00000000	$-0.78539816 \approx -\pi/4$	$1.41421354 \approx \sqrt{2}$	
$-5.96856597e - 7$	-1.00911547	$-2.60499122e - 5$	$-1.81893539e - 5$
$1.22755482e - 6$	1.87773726	$-4.76966579e - 5$	
$4.59348246e - 8$	-6.38405893	$1.77429347e - 5$	

are qualitatively satisfactory and free of numerical artifacts.

Remark 2. We note a limitation of our model, which is that the loss function in Equation (5) requires the minimal period T of the data. This can be either identified from the dataset or, for an analytic signal as used here for validation, directly from the mathematical expression.

3.1 | Analytic periodic functions

The design of the FNN lends itself naturally to simple linear combinations of Fourier modes. We first assess a function of period $T = 2$ given by

$$f(x) = \cos(\pi x) + \sin(\pi x). \quad (14)$$

Although this function has only one mode, we build the network with four nodes in the hidden layer and seek

to observe whether the redundant nodes converge to zero. To investigate the impact of different regularization terms in the loss function, we consider both L^2 and L^1 norms in Equation (5). We report the network converged parameters in Tables 1 and 2, along with the number of iterations and the minimized loss function. Analytical work indicates that the expected output of the FNN is

$$\hat{u}(x) \approx \sqrt{2} \cos\left(\pi x - \frac{\pi}{4}\right),$$

which is the reduced form of $f(x) = \cos(\pi x) + \sin(\pi x)$. The optimization converged to approximately $2e - 4$ in 189 iterations for the L^2 regularization, as illustrated in Table 1. The convergence rate is faster for the L^1 regularization, providing in 87 iterations a loss function of magnitude order $3e - 4$, as in Table 2. Although the L^1 regularization displays better convergence properties, the weights themselves do not converge to exactly zero as expected (see the weights corresponding to nodes 2–4 in Table 2).

Number of iterations			87
Loss function (upon convergence)			$3e-4$
w_k	ϕ_k	λ_k	ϕ_0
1.00000033	$-0.78540193 \approx -\pi/4$	$1.41421847 \approx \sqrt{2}$	
1.4969094	-0.71483098	$-4.32639025e-6$	$1.58219741e-5$
0.05095418	0.96613253	$6.73308554e-5$	
0.14126515	-5.84166681	$-5.13186621e-5$	

TABLE 2 Number of iterations, value of the loss function at convergence, and optimal weights and biases of the FNN to approximate $f(x) = \cos(\pi x) + \sin(\pi x)$, $k = 1, \dots, 4$ with a L^1 regularization

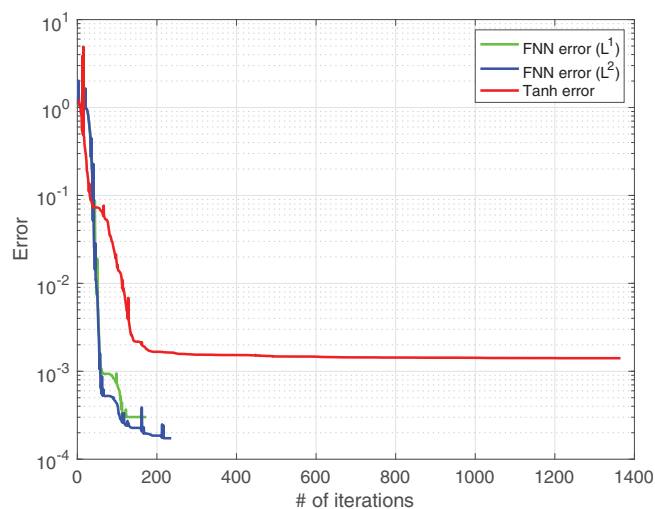


FIGURE 4 Comparison in terms of the values of the loss function throughout the optimization when using an FNN and when using a neural network with a tanh activation

In Figure 4 we compare the performance of the newly constructed FNN with a neural network with a tanh activation function and a Glorot initialization, as in [9]. The tanh activation function yields a slow convergence of 1173 iterations to a loss function of $1e-3$, considerably inferior to the performance of the FNN in terms of both efficiency and accuracy. Beyond performance the FNN has the added benefit of producing an approximation valid outside the training region. In Figure 5 we extend the testing domain and note that periodicity is preserved for the FNN model, while the tanh network loses extensibility. The inability to preserve the sought function outside of the training region is a typical shortcoming of neural network methods for function approximation. The FNN model devised here is able to overcome this issue by imposing periodicity in the loss function, defined in Equation (5), as well as by using a periodic activation function. We note that we did not include the periodicity requirement in the loss function when training the traditional neural network. The results were worse in that this required more nodes in the hidden

layer; convergence was reached more slowly; and the error was similar. Furthermore, the traditional network did not preserve the sought function properties outside of $[-1 \pm T, 1 \pm T]$ when the periodicity requirement was added.

We now consider a function with more modes and also higher amplitudes:

$$g(x) = 8 \cos(4\pi x) + \sin(2\pi x) + \sin(\pi x). \quad (15)$$

We show in Tables 3 and 4 the value of the loss function at convergence, the number of iterations, and the values of the converged weights. The optimization converged to approximately $9e-4$ after 195 iterations using a L^2 regularization and to approximately $1e-3$ after 169 iterations using the L^1 norm. We note that the biases are approximations of odd multiples of $\pi/2$ for the sine part of the function g and approximately 0 for its cosine component.

In Figure 6 we illustrate the comparison between the exact function g , the approximations provided by the FNN and by the tanh network. The actual error between the function and the FNN approximation has an order of magnitude $1e-8$ for the L^2 regularization and $1e-5$ for the L^1 one. Performance, as well as the lack of convergence of redundant nodes to zero using L^1 regularization, motivates us to use L^2 regularization in the remainder of the paper. Comparing the FNN approximation with its counterpart obtained by using a tanh neural network, we note again a poor convergence for the tanh activation function. The local minimum for a four-node hidden layer using the tanh activation has an order of magnitude 1, implying lack of convergence, while for 13 nodes in the hidden layer convergence is attained at a value of $1e-2$ for the loss function. The FNN approximates the sought function outside the training region while the tanh does not. We note that we sampled the same number of points on the training interval for the function g in Equation (15) as for the function f defined in Equation (14), which has only one mode.

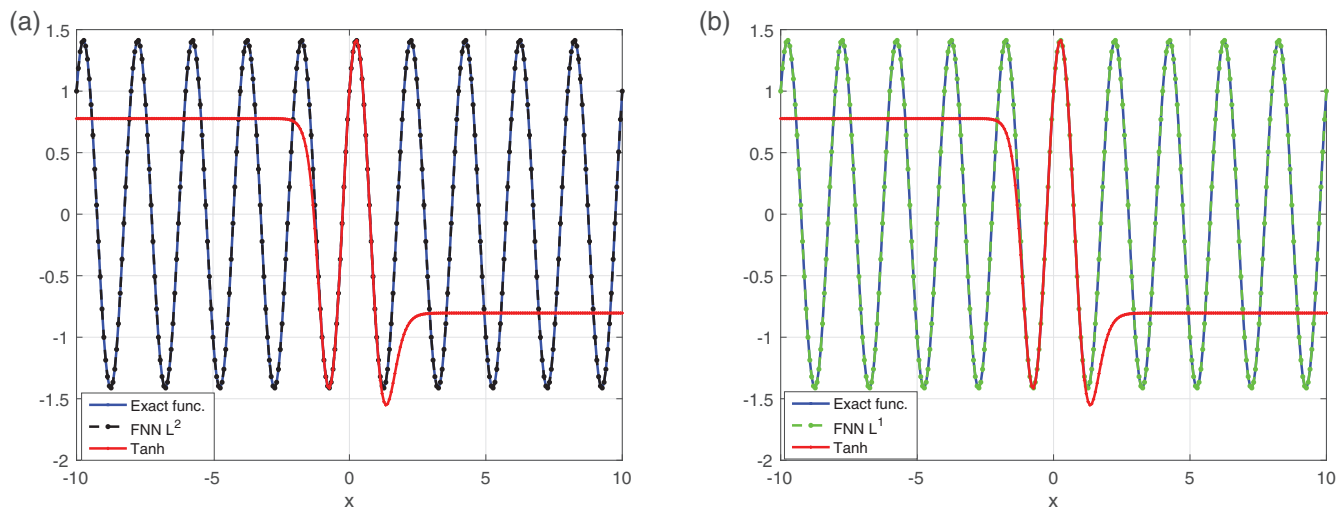


FIGURE 5 Comparison between $f(x) = \cos(\pi x) + \sin(\pi x)$ and the output of the FNN outside of the training domain for different regularization norms. (a) L^2 regularization. (b) L^1 regularization

TABLE 3 Number of iterations, value of the loss function at convergence, and optimal weights and biases of the FNN to approximate $g(x) = 8 \cos(4\pi x) + \sin(2\pi x) + \sin(\pi x)$ with L^2 regularization, $k = 1, \dots, 4$

Number of iterations			195
Loss function (upon convergence)			$9e - 4$
w_k	ϕ_k	λ_k	ϕ_0
1.00000000	$1.57079627 \approx \pi/2$	$-9.99999993e - 1$	
$5.40604942e - 6$	$1.01888743e1$	$3.34351764e - 6$	$2.41330937e - 6$
4.00000000	$-9.14402304e - 10$	7.99999983	
-2.00000000	$4.71238899 \approx 3\pi/2$	$-9.99999984e - 1$	

TABLE 4 Number of iterations, value of the loss function at convergence, and optimal weights and biases of the FNN to approximate $g(x) = 8 \cos(4\pi x) + \sin(2\pi x) + \sin(\pi x)$ with L^1 regularization, $k = 1, \dots, 4$

Number of iterations			169
Loss function (upon convergence)			$1e - 3$
w_k	ϕ_k	λ_k	ϕ_0
0.99999306	$1.57072838 \approx \pi/2$	-1.000141261	
1.99999672	$7.85406634 \approx 5\pi/2$	$-9.99825909e - 1$	$3.82384745e - 5$
3.99999998	$-1.14376588e - 5$	7.99989569	
-0.04032155	$-2.12089270e + 1$	$1.69927207e - 5$	

3.2 | Piecewise continuous periodic functions

We now seek to approximate nonperiodic functions, which are periodically extended through continuity. The parabola

$$f(x) = x^2, \quad x \in (-(2k+1), (2k+1)), k \in \mathbf{N} \quad (16)$$

has the Fourier series

$$S_N f(x) = \frac{1}{3} + \sum_{n=1}^N 4 \frac{(-1)^n}{\pi^2 n^2} \cos(\pi n x).$$

Using four nodes in the hidden layer, the FNN captures the first five modes of the Fourier decomposition (if we count the zeroth mode). We show in Table 5 the

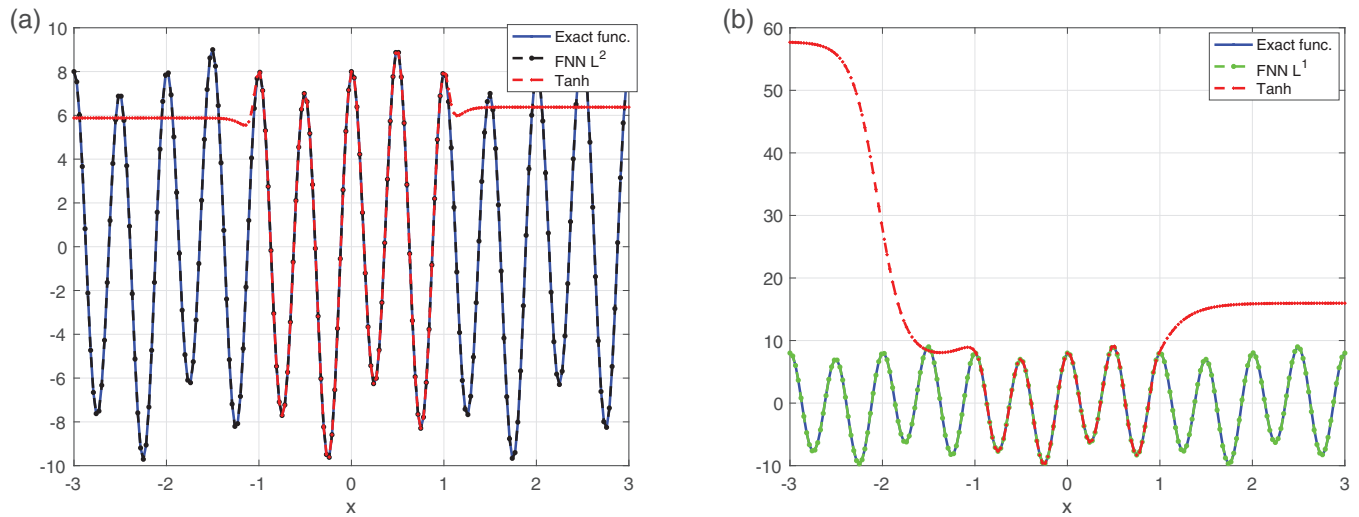


FIGURE 6 Comparison between $g(x) = 8 \cos(4\pi x) + \sin(2\pi x) + \sin(\pi x)$ and the output of the FNN for different regularization norms. (a) L^2 regularization. (b) L^1 regularization

values obtained for the weights and the biases. We denote by FNN coefficients the hidden layer to output weights and the bias of the output layer. We note that they are approximations of the Fourier coefficients with an order of magnitude of $1e-3$. Figure 8a shows an error bar plot for the exact Fourier coefficients against the error from the FNN coefficients and provides a qualitative assessment of the approximation.

We illustrate in Figure 7 the behavior of the FNN output against the approximation from a neural network with a tanh activation function. In this case as well, the FNN network conserves the properties of the function to be approximated outside of the training domain, while the tanh neural network does not. We note that the latter was more accurate in this case on the training interval $[-1, 1]$ ($6e-4$ with 3073 iterations against $2e-2$ with 130 iterations for the FNN).

We also assessed the function

$$g(x) = |x|, \quad x \in (-(2k+1), (2k+1)), \quad k \in \mathbf{N}, \quad (17)$$

which has the Fourier series

$$S_N g(x) = \frac{1}{2} + \sum_{n=1}^N -4 \frac{1}{\pi^2(2n-1)} \cos(\pi(2n-1)x).$$

We summarize the FNN coefficients in Table 6. The FNN was able to capture the first four modes of the Fourier decomposition and split the value of the third mode between two nodes in the hidden layer as showcased in Table 6. The reason is that g admits Fourier coefficients only for odd coefficients. We provide in Figure 8b a visual representation to compare the Fourier coefficients of g

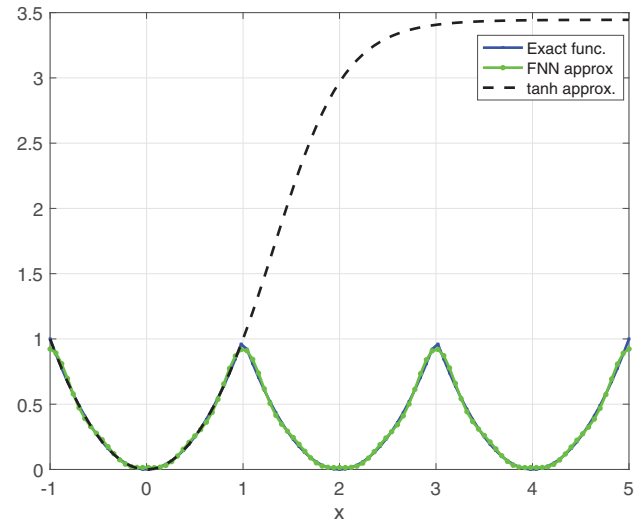


FIGURE 7 Comparison between the FNN and the tanh approximations outside the training domain for $f(x) = x^2$

with its FNN coefficients. Results similar to those obtained for the function f defined in Equation (16) are observed here. The FNN coefficients are good approximations of their Fourier counterparts, and the error between the two has $1e-3$ order of magnitude.

4 | FNNs FOR DIFFERENTIAL EQUATIONS SOLVERS

ML in the field of differential equations fills various gaps, from providing alternative models to the classical continuum mechanics equations, to inverse problem solvers, and

TABLE 5 Number of iterations, value of the loss function at convergence, and optimal weights and biases of the FNN to approximate $f(x) = x^2$ on $[-1, 1]$

Number of iterations			130
Loss function at convergence			$2e - 2$
w_k	ϕ_k	λ_k	ϕ_0
0.99995578	-0.00477923	-0.40479907	
2.99892898	-3.139197051	0.04915202	0.335023246
3.99604127	0.01794715	0.02874206	
-1.99965386	0.00445702	0.10497063	

TABLE 6 Number of iterations, value of the loss function at convergence, and optimal weights and biases of the FNN to approximate $f(x) = |x|$ on $[-1, 1]$

Number of iterations			445
Loss function at convergence			$1e - 2$
w_k	ϕ_k	λ_k	ϕ_0
0.99995402	$-2.44738021e - 3$	-0.40420162	
2.98845687	$-1.68031025e - 1$	0.03295792	0.502531
2.99445216	$-6.78306499e - 2$	-0.07711458	
-1.99075052	-9.45244148	-0.00391551	

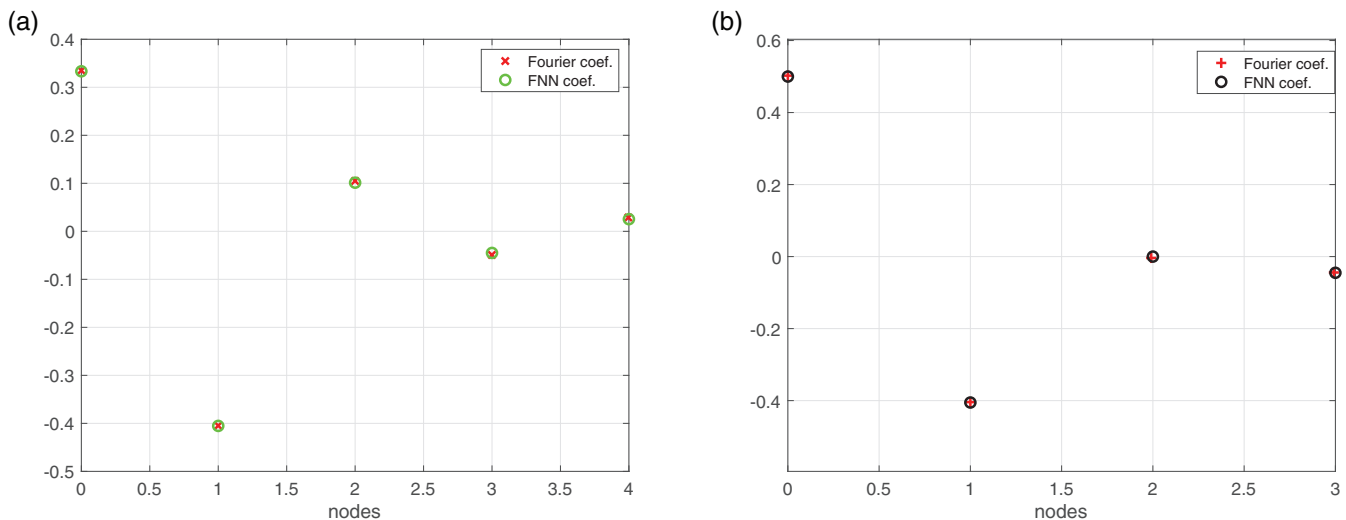


FIGURE 8 Comparison between the FNN coefficients and the Fourier coefficients corresponding to the nodes captured by the neural network for the functions studied. (a) $f(x) = x^2$. (b) $g(x) = |x|$

thus has generated considerable interest in recent years. ML techniques have been used not only to effectively solve differential equations [6, 13, 26, 27, 33] but also to discover underlying dynamics from data [3, 25, 29]. A different approach is to build robust neural networks architectures based on differential equations [4, 20, 31].

In this section, we extend the work presented in Section 2 to compute periodic solutions of differential equations of the type $Pu = f$, where P is a differential

operator. To this end, we follow [25, 33] and set the loss function to be the residual of the differential equation we intend to solve. In a spirit similar to the weak formulations for finite element solutions [14], the solution is obtained by minimizing $\|P\hat{u}(x) - f(x)\|_2^2$. The concept was introduced in [33] and was called PINNs in [27]. To leverage the FNNs introduced here, we append to the loss function the periodicity requirement as well as the regularization terms to obtain

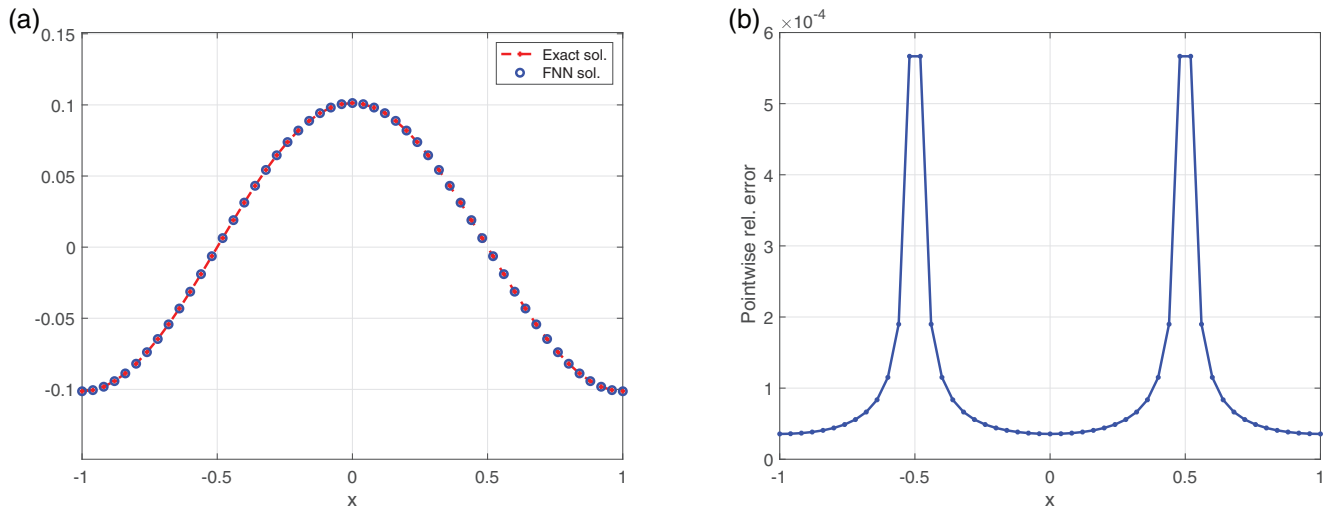


FIGURE 9 Qualitative assessment of the PIFNN solution for the Poisson equation. (a) Comparison between the PIFNN and the exact solutions. (b) Pointwise relative error between the PIFNN and the exact solutions

w_k	ϕ_k	λ_k	ϕ_0
$-3.77424453e-10$	-4.42730681	$4.90842033e-5$	
$-1.16548318e-10$	2.46210794	$-1.45660922e-5$	0
1.00000000	$3.14159265 \approx \pi$	$-1.01321184e-1 \approx -1/\pi^2$	
$-1.18183192e-9$	0.79153364	$-1.60990031e-6$	

TABLE 7 Optimal weights and biases of the PIFNN to solve the Poisson equation (19), $k = 1, \dots, 4$

$$\begin{aligned}
 L(\phi, w, \lambda) = & \|P\hat{u}(x) - f(x)\|_2^2 + \alpha_2 \|\lambda\|_2^2 + \alpha_3 \|w\|_2^2 \\
 & + \alpha_3 (\|\hat{u}(x+T) - \hat{u}(x)\|_2^2) \\
 & + \alpha_4 (\|\hat{u}(x-T) - \hat{u}(x)\|_2^2).
 \end{aligned} \quad (18)$$

We refer to this residual-based loss function approach in conjunction with cosine as the activation function as PIFNN, to extend on previous work.

4.1 | Poisson equation

Let us consider the Poisson equation with periodic boundary conditions,

$$-\Delta u = \cos(\pi x), \quad x \in (-1, 1), \quad \text{with } u(-1) = u(1), \quad (19)$$

and compute its solution using the constructed PIFNN with four nodes in the hidden layer. The exact solution of the equation, given by $u = \frac{1}{\pi^2} \cos(\pi x)$, allows us to assess the accuracy of the PIFNN solution. We show both solutions in Figure 9a, and we report the weights and biases of the PIFNN in Table 7. We note that the output is

$$\hat{u}(x) = -\frac{1}{\pi^2} \cos(\pi x + \pi) + o(10^{-5}) \approx \frac{1}{\pi^2} \cos(\pi x).$$

and we observe that the pointwise relative error between the two solutions has order of magnitude $1e-4$ in Figure 9b. In contrast with the original PINN approach that provides an uninterpretable neural network model for the solution of the differential equation, PIFNN provides an analytic formula fully determined by the computed weights.

4.2 | Heat equation

To extend further the FNN framework to time-dependent problems, we consider the heat equation with periodic boundary conditions over an interval $[-1, 1]$. The solution is therefore $\tau = 2$ periodic:

$$\begin{aligned}
 \frac{\partial u}{\partial t} &= \frac{\partial^2 u}{\partial x^2}, \quad x \in (-1, 1), \quad t \in [0, 4], \\
 u(0, x) &= \sin(\pi x), \quad u(t, -1) = u(t, 1).
 \end{aligned} \quad (20)$$

Various strategies are available for treating time-dependent PDEs by using neural networks [27, 33]. Here we suggest an alternative idea to account for the time dependency, namely, to construct two separate networks

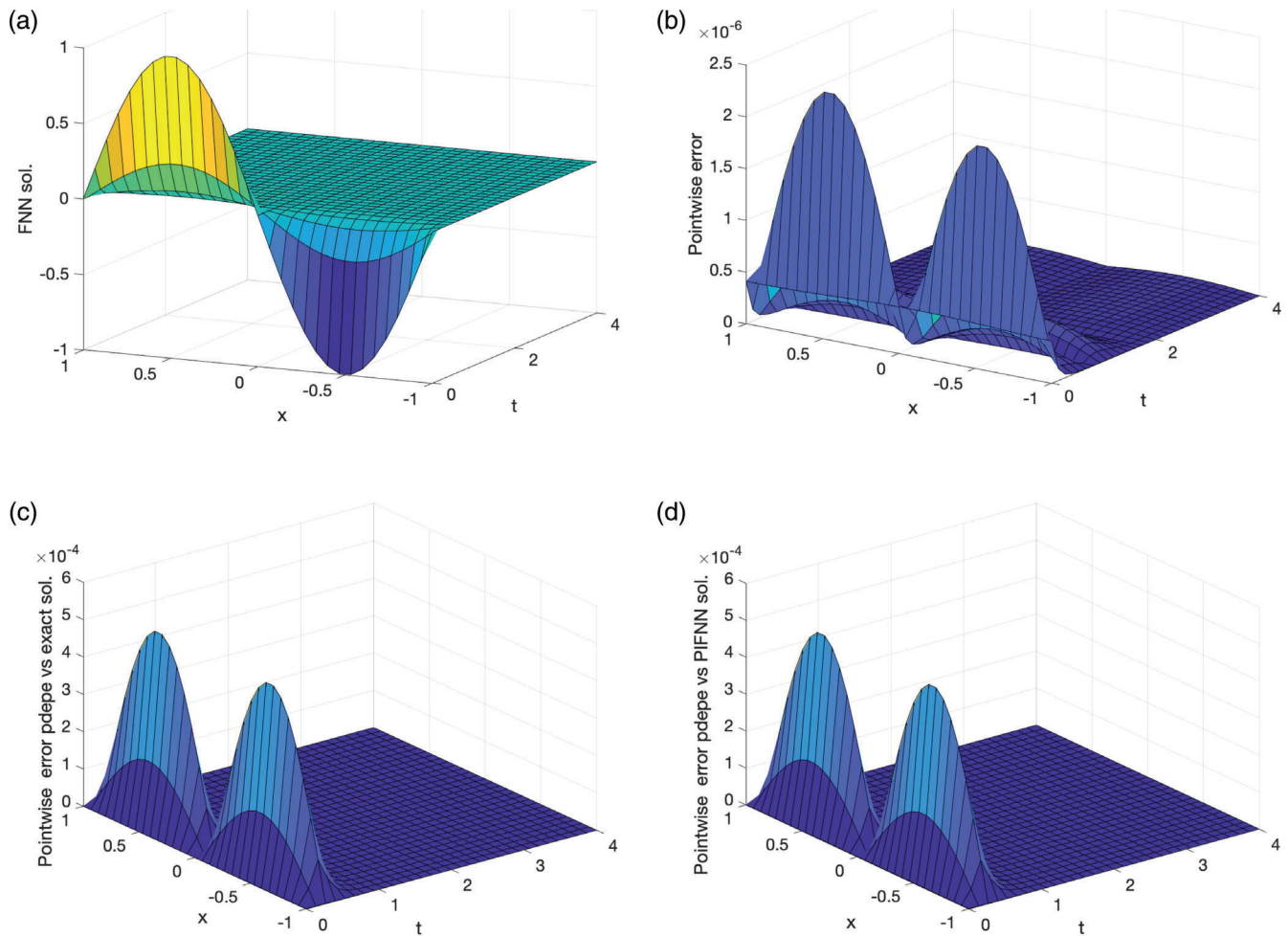


FIGURE 10 Qualitative assessment of the PIFNN solution for the heat equation. (a) PIFNN solution. (b) Pointwise error between the PIFNN and the exact solutions. (c) Pointwise error between the PDEPE and the exact solutions. (d) Pointwise error between the PIFNN and the PDEPE solutions

for the spatial and time dimension. This can be achieved by rewriting the heat equation using the separation of variables method. To this end, we introduce two separate variables X, T for the spatial and time component, respectively, and set $u(x, t) = X(x)T(t)$. Replacing the ansatz on u in Equation (20) transforms the equation into

$$X(x)T'(t) = X''(x)T(t).$$

Note that this strategy is not universally applicable, but it is a common approach used to derive analytical solutions to linear differential equations. We rewrite the cost function as

$$\begin{aligned} L(\phi, w, \lambda) = & \alpha_1 \|\hat{X}(x)\hat{T}'(t) - \hat{X}''(x)\hat{T}(t)\|^2 + \alpha_2 \|\lambda\|^2 \\ & + \alpha_3 \|w\|^2 + \alpha_3 \|\hat{X}(x+2) - \hat{X}(x)\|^2 \\ & + \alpha_4 \|\hat{X}(x-2) - \hat{X}(x)\|^2 + \alpha_5 \|\hat{T}_0(x) - \hat{u}_0(x)\|^2, \end{aligned}$$

to incorporate the boundary and initial conditions and separate the network into two separate subnetworks. The spatial components are modeled by using the PIFNN approach with one node in the hidden layer, while the time component is treated by using a shallow neural network with a tanh activation function and five nodes in the hidden layer.

Figure 10 provides a qualitative assessment of the PIFNN solution for the heat equation (20). Figure 10a corresponds to the PIFNN solution and Figure 10b the error between that solution and the exact one. We appended a comparison with the solution to Equation (20) provided by the Matlab *pdepe* solver [21]. The error between the PIFNN and the exact solution is of magnitude $1e-6$ and the one between the *pdepe* solution and the exact one is of magnitude $1e-4$. The model chosen here is likely to ensure smoothness given that errors in Fourier coefficients are mollified by the activation function.

5 | CONCLUSION

We have constructed a novel FNN model that successfully approximates low-frequency analytic and piecewise continuous one-dimensional periodic functions. By combining the FNN with a residual-based loss function, as in the PINN model, we define a fully interpretable network for differential equations with periodic boundary conditions, which we refer to as PIFNN. The FNN model replaces commonly used activation functions, such as tanh or sigmoid, by a periodic function. With cosine as the activation function, the network model mimics a Fourier decomposition. To ensure the periodic nature is preserved, we appended to the loss function the requirement that the solution be periodically replicated also outside the training domain. Besides significantly improving the results obtained using traditional neural networks in terms of accuracy and number of iterations, our simulations revealed another significant advantage: namely, FNN conserves the properties of the learned task (approximating a function, etc.) outside the training domain.

One limitation of this model is that we do not use a rigorous technique for picking the penalty coefficients but set them heuristically. Another limitation is that we need to know the period of the function we estimate a priori to incorporate it in the activation function and loss function. We are currently investigating solutions to overcome these limitations. Our results are promising, and we plan on extending them to multidimensional functions and nonlinear differential equations. We believe the framework presented here can serve as a pre- and postprocessing tool in many engineering and scientific applications, where the Fourier decomposition is widely used, such as electronics, image recognition, and acoustics.

ACKNOWLEDGMENTS

We thank Charlotte Haley for her pertinent suggestions. This work was supported by the U.S. Department of Energy, Office of Science, Office of Advanced Scientific Computing Research, Scientific Discovery through Advanced Computing (SciDAC) program and the FAST-Math Institute under Contract No. DE-AC02-06CH11357 at Argonne National Laboratory. *Government License*. The submitted manuscript has been created by UChicago Argonne, LLC, Operator of Argonne National Laboratory ("Argonne"). Argonne, a U.S. Department of Energy Office of Science laboratory, is operated under Contract No. DE-AC02-06CH11357. The U.S. Government retains for itself, and others acting on its behalf, a paid-up nonexclusive, irrevocable worldwide license in said article to reproduce, prepare derivative works, distribute copies to the public, and perform publicly and display publicly, by or on behalf of the Government. The Department of

Energy will provide public access to these results of federally sponsored research in accordance with the DOE Public Access Plan. <http://energy.gov/downloads/doe-public-access-plan>.

DATA AVAILABILITY STATEMENT

The data that support the findings of this study are available from the corresponding author upon request.

ORCID

Marieme Ngom  <https://orcid.org/0000-0002-3240-569X>

REFERENCES

1. A. Barron, *Universal approximation bounds for superpositions of a sigmoidal function*, in *Machine Learning and Cybernetics*, 39 (1993), 930–945.
2. D. Benson, *Music: A mathematical offering*, Cambridge University Press, Cambridge, 2013.
3. S. L. Brunton, J. L. Proctor, and N. Kutz, *Discovering governing equations from data by sparse identification of nonlinear dynamical systems*, *Proc. Natl. Acad. Sci. U. S. A.* 113 (2016), 3932–3937.
4. T. Q. Chen, Y. Rubanova, J. Bettencourt, and D. Duvenaud, *Neural ordinary differential equations*, 32nd Conference on Neural Information Processing Systems, 2018.
5. G. Cybenko, *Approximation by superpositions of a sigmoidal function*, *Math. Control Signals Syst.* 2 (1989), 303–314.
6. T. Dockhorn, *A discussion on solving partial differential equations using neural networks*, <http://arxiv.org/abs/1904.07200>, 2019.
7. R. Gallant and H. White, *There exists a neural network that does not make avoidable mistakes*, *Proceedings of the Second Annual IEEE Conference on Neural Networks*, 1988.
8. A. Geron, *Hands-on machine learning with scikit-learn and tensorflow*, O'Reilly Media, Inc, USA, 2017.
9. X. Glorot and Y. Bengio, *Understanding the difficulty of training deep feedforward neural networks*, *Neural Netw.* 9 (2010), 249–256.
10. K. He, X. Zhang, S. Ren, and J. Sun, *Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification*, *Proceedings of the 2015 IEEE International Conference on Computer Vision*, 2015.
11. S. Hochreiter and J. Schmidhuber, *Long short-term memory*, *Parallel Distributed Processing, Neural Comput.* 9 (1986), 1735–1780.
12. K. Hornik, M. Stinchcombe, and H. White, *Multilayer feed-forward networks are universal approximators*, *Neural Netw.* 2 (1989), 359–366.
13. J.-T. Hsieh, S. Zhao, S. Eismann, L. Mirabella, and S. Ermon, *Learning neural PDE solvers with convergence guarantees*, *International Conference on Learning Representations 2019*, 2019.
14. D. Hutton, *Fundamentals of finite element analysis*, McGraw-Hill, New York, NY, 2004.
15. S. K. Kumar, *On weight initialization in deep neural networks*, <http://arxiv.org/abs/1704.08863>, 2017.
16. Y. LeCun, P. Haffner, L. Bottou, and Y. Bengio, *Object recognition with gradient-based learning*, *International Workshop on Shape, Contour and Grouping in Computer Vision*, 1999.

17. M. Leshno, V. Lin, A. Pinkus, and S. Schocken, *Multilayer feedforward networks with a nonpolynomial activation function can approximate any function*, Neural Netw. 6 (1993), 861–867.
18. Z. Li, N. Kovachki, K. Azizzadenesheli, B. Liu, K. Bhattacharya, A. Stuart, and A. Anandkumar, *Fourier neural operator for parametric partial differential equations*, 2021.
19. S. Liu, *Fourier neural network for machine learning*, 2013 International Conference on Machine Learning and Cybernetics, 2013.
20. Z. Long, Y. Lu, X. Ma, and B. Dong, *PDE-NET: Learning PDEs from data*, Proceedings of the 35th International Conference on Machine Learning, 2019.
21. MATLAB, 9.6.0.1135713 (R2019a), The MathWorks Inc, Natick, MA, 2018.
22. C. Nwankpa, W. Ijomah, A. Gachagan, and S. Marshall, *Activation functions: Comparison of trends in practice and research for deep learning*. <http://arxiv.org/abs/1811.03378>, 2018.
23. G. Parascandolo, H. Huttunen, and T. Virtanen, *Taming the waves: Sine as activation function in deep neural networks*, <https://openreview.net/pdf?id=Sks3zF9eg>, 2017.
24. W. Popiński, *On least squares estimation of Fourier coefficients and of the regression function*, Appl. Math. 22 (1993), 91–102.
25. M. Raissi, *Deep hidden physics models: Deep learning of nonlinear partial differential equations*, J. Mach. Learn. Res. 19 (2018), 1–24.
26. M. Raissi and G. E. Karniadakis, *Hidden physics models: Machine learning of nonlinear partial differential equations*, J. Comput. Phys. 348 (2017), 683–693.
27. M. Raissi, P. Perdikaris, and G. E. Karniadakis, *Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations*, J. Comput. Phys. 378 (2019), 686–707.
28. S. Ross, *Applied probability models with optimization applications*, Dover Publications Inc., Mineola, NY, 1992.
29. S. H. Rudy, S. L. Brunton, J. L. Proctor, and N. Kutz, *Data-driven discovery of partial differential equations*, Sci. Adv. 3 (2017), e1602614.
30. D. E. Rumelhart, G. E. Hinton, and R. J. Williams, *Learning internal representations by error propagation*, Readings in Cognitive Science, A Perspective from Psychology and Artificial Intelligence, Morgan Kaufmann, USA, (1998).
31. L. Ruthotto and E. Haber, *Deep neural networks motivated by partial differential equations*, J. Math. Imaging Vis. 62 (2020), 352–364.
32. A. Silvescu, *Fourier neural networks*, International joint conference on neural networks, 1999.
33. J. Sirignano and K. Spiliopoulos, *DGM: A deep learning algorithm for solving partial differential equations*, J. Comput. Phys. 375 (2018), 1339–1364.
34. A. Zhumekenov, M. Uteuliyeva, O. Kabdolov, R. Takhanov, Z. Assylbekov, and A. J. Castro, *Fourier neural networks: A comparative study*. <http://arxiv.org/abs/1902.03011>, 2019.

How to cite this article: M. Ngom, and O. Marin, *Fourier neural networks as function approximators and differential equation solvers*, Stat. Anal. Data Min.: ASA Data Sci. J. **14** (2021), 647–661. <https://doi.org/10.1002/sam.11531>

APPENDIX A.

Statistical properties

We recall below some properties verified by the mean μ and the variance σ^2 of a random variable.

- 1 $\mu(aX + Y) = a\mu(X) + \mu(Y)$ and $\sigma^2(aX + Y) = a^2\sigma^2(X) + \sigma^2(Y)$, where $a \in \mathbf{R}$ and X and Y are two random variables.
- 2 $\mu(XY) = \mu(X)\mu(Y)$ and $\sigma^2(XY) = \sigma^2(X)(\mu^2(Y) + \sigma^2(Y)) + \mu^2(X)\sigma^2(Y)$, where X and Y are two independent random variables.

We also recall the value of the mean and of the variance of the two distributions used in the paper, namely, the uniform and normal distributions.

- 1 If $X \sim \mathcal{U}([a, b])$ then

- $\mu(X) = \frac{1}{b-a}$ and $\sigma^2(X) = \frac{(b-a)^2}{12}$.
- The probability density function (pdf) of X is $f_X(x) = \frac{1}{b-a}$ if $x \in [a, b]$ and 0 otherwise.

- 2 If $X \sim \mathcal{N}(0, m^2)$, then

- $\mu(X) = 0$ and $\sigma^2(X) = m^2$,
- The probability density function (pdf) of X is $f_X(x) = \frac{1}{\sigma\sqrt{2\pi}}e^{-\frac{x^2}{2\sigma^2}}$.